

1. What is Recursion?

Definition:

Recursion is a programming technique where a **function calls itself** directly or indirectly until a base condition is met.

Real-life analogy:

Imagine you have a set of Russian dolls.

To open the biggest one, you open it → find a smaller doll → open that → and so on...

Finally, you reach the smallest doll that cannot be opened (base case).

Similarly, in recursion:

- You break the big problem into smaller sub-problems.
- Each smaller sub-problem is solved by calling the same function again.
- You stop when you reach a **base case**.

2. Structure of a Recursive Function

```
void recursiveFunction() {  
    // Base case (stopping condition)  
    if (condition) return;  
  
    // Recursive call (smaller problem)  
    recursiveFunction();  
}
```

3. Principles of Recursion

1. **Base Case:**
The condition that stops the recursion.
Without this, recursion will go infinite → `StackOverflowError`.
2. **Recursive Case:**
The step where the function calls itself with a smaller/simpler input.
3. **Stack Behavior:**
Recursion uses a **function call stack** → last function called is the first to complete (LIFO).

4. Tail Recursion

A **tail recursive** function is one where **the recursive call is the last thing** executed in the function.

✓ **Tail recursion is memory efficient** because compiler can optimize it (no need to keep previous function calls in stack).

Example — Normal Recursion (Non-tail)

```
int sum(int n) {  
    if (n == 0) return 0;  
    return n + sum(n - 1); // <- work happens after recursive call  
}
```

Example — Tail Recursion

```
int sumTail(int n, int acc) {  
    if (n == 0) return acc;  
    return sumTail(n - 1, acc + n); // <- recursive call is last statement  
}
```

To call:

```
System.out.println(sumTail(5, 0)); // Output: 15
```

5. Fibonacci Numbers using Recursion

Fibonacci Series:

0, 1, 1, 2, 3, 5, 8, 13, ...

Formula:

$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

with base cases $\rightarrow \text{fib}(0)=0, \text{fib}(1)=1$

Code:

```
class Main {  
    static int fib(int n) {  
        if (n == 0) return 0; // base case 1  
        if (n == 1) return 1; // base case 2  
        return fib(n - 1) + fib(n - 2); // recursive case  
    }  
  
    public static void main(String[] args) {  
        int n = 6;  
        System.out.println("Fibonacci of " + n + " is: " + fib(n));  
    }  
}
```

 **Output:**

Fibonacci of 6 is: 8

6. Power of a number (a^n)

We can define:

$a^n = a \times a^{n-1}$
Base case: $a^0 = 1$

Code:

```
class Main {
    static int power(int a, int n) {
        if (n == 0) return 1;          // base case
        return a * power(a, n - 1);    // recursive case
    }

    public static void main(String[] args) {
        System.out.println("2^5 = " + power(2, 5));
    }
}
```

Output:

2^5 = 32

Optimized Version (Divide and Conquer)

To reduce recursive calls:

$a^n = (a^{(n/2)})^2$ if n is even
 $a^n = a \times (a^{(n/2)})^2$ if n is odd

Code:

```
class Main {
    static int powerFast(int a, int n) {
        if (n == 0) return 1;
        int half = powerFast(a, n / 2);
        if (n % 2 == 0)
            return half * half;
        else
            return a * half * half;
    }

    public static void main(String[] args) {
        System.out.println("2^10 = " + powerFast(2, 10));
    }
}
```

Output:

$$2^{10} = 1024$$

7. Reverse of a String using Recursion

Logic:

- Take first character and move it to the end.
- Repeat for the rest of the string.

Code:

```
class Main {
    static String reverse(String s) {
        if (s.isEmpty()) return s; // base case
        return reverse(s.substring(1)) + s.charAt(0); // recursive case
    }

    public static void main(String[] args) {
        System.out.println(reverse("hello"));
    }
}
```

Output:

olleh

Summary Table

Problem	Base Case	Recursive Case	Example Output
Sum	$n == 0$	$n + \text{sum}(n-1)$	$\text{sum}(5)=15$
Fibonacci	$n == 0, n == 1$	$\text{fib}(n-1) + \text{fib}(n-2)$	$\text{fib}(6)=8$
Power	$n == 0$	$a * \text{power}(a, n-1)$	$2^5=32$

Reverse String empty string reverse(rest)+first char "hello" → "olleh"

◆ 1. Head Recursion (Work after recursive call)

- 👉 In head recursion, the recursive call happens **first**, and then we do our work **after returning**.
- 👉 For factorial, we let recursion compute smaller factorial, then multiply.

C++ (Head Recursion)

```
#include <iostream>
using namespace std;

int factorial_head(int n) {
    if (n == 0 || n == 1) return 1;    // base case
    int smaller = factorial_head(n - 1); // recursive call first
    return n * smaller;                // work after recursion returns
}

int main() {
    cout << "Factorial (Head Recursion) of 5: " << factorial_head(5) << endl;
    return 0;
}
```

Java (Head Recursion)

```
public class FactorialHead {
    static int factorial_head(int n) {
        if (n == 0 || n == 1) return 1;    // base case
        int smaller = factorial_head(n - 1); // recursive call first
        return n * smaller;                // work after recursion
    }

    public static void main(String[] args) {
        System.out.println("Factorial (Head Recursion) of 5: " +
            factorial_head(5));
    }
}
```

◆ 2. Tail Recursion (Work before recursive call, result carried forward)

- 👉 In tail recursion, the function does its **work first** and passes the result along.
- 👉 Here we use an **accumulator** parameter.

C++ (Tail Recursion)

```
#include <iostream>
using namespace std;

int factorial_tail(int n, int result = 1) {
    if (n == 0 || n == 1) return result;    // base case
    return factorial_tail(n - 1, n * result); // work done before recursion
}

int main() {
    cout << "Factorial (Tail Recursion) of 5: " << factorial_tail(5) << endl;
    return 0;
}
```

Java (Tail Recursion)

```
public class FactorialTail {
    static int factorial_tail(int n, int result) {
        if (n == 0 || n == 1) return result;    // base case
        return factorial_tail(n - 1, n * result); // pass result forward
    }

    public static void main(String[] args) {
        System.out.println("Factorial (Tail Recursion) of 5: " +
            factorial_tail(5, 1));
    }
}
```



1. Recursive Implementation of `atoi()`



Problem Statement

Convert a string (which may have digits and optional sign +/-) into an integer recursively.

Ex: "1234" → 1234, "-42" → -42.



Pattern

String → Integer using recursion (process one char, delegate rest).



Trick to Remember

👉 Think: **Last char handles smallest digit, recursion handles prefix.**

- Base: empty string → 0.
- Recursive: `result = atoi(str[0..n-2]) * 10 + (lastChar - '0')`.

Variations

- Implement `itoa()` (reverse process).
- Handle spaces, invalid chars.
- Base conversion (binary $\rightarrow$ decimal).

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

int myAtoi(string s, int n, int i=0, int sign=1) {
    if(i==n) return 0;
    if(i==0 && (s[i]=='-' || s[i]=='+'))
        return myAtoi(s, n, i+1, s[i]=='-'?-1:1);
    return sign * ((s[i]-'0')*pow(10,n-i-1) + abs(myAtoi(s,n,i+1,sign)));
}

int main(){
    string s = "-1234";
    cout << myAtoi(s, s.size()) << endl; // -1234
}
```

Complexity

- **Time:** $O(n)$
 - **Space:** $O(n)$ recursion depth
-

2. Pow(x, n) – Fast Exponentiation

Problem Statement

Compute x^n (where n can be large, positive/negative).

Pattern

Divide & Conquer recursion $\rightarrow$ split power into halves.

Trick to Remember

👉 “Even exponent halves, odd exponent reduces by one.”

- If n even $\rightarrow \text{pow}(x, n) = \text{pow}(x, n/2) * \text{pow}(x, n/2)$.

- If n odd $\rightarrow x * \text{pow}(x, n-1)$.
- Handle negative n .

Variations

- Modular exponentiation (very common in CP).
- Matrix exponentiation (for Fibonacci fast calculation).

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

double myPow(double x, long long n){
    if(n==0) return 1;
    if(n<0) return 1.0/myPow(x,-n);
    double half = myPow(x,n/2);
    return (n%2==0) ? half*half : half*half*x;
}

int main(){
    cout << myPow(2.0, 10) << endl; // 1024
    cout << myPow(2.0, -3) << endl; // 0.125
}
```

Complexity

- **Time:** $O(\log n)$
- **Space:** $O(\log n)$ recursion

3. Count Good Numbers

Problem Statement

A digit string is **good** if the digits (**0-indexed**) at **even** indices are **even** and the digits at **odd** indices are **prime** (2, 3, 5, or 7).

- For example, "2582" is good because the digits (2 and 8) at even positions are even and the digits (5 and 2) at odd positions are prime. However, "3245" is **not** good because 3 is at an even index but is not even.

Given an integer n , return *the **total** number of good digit strings of length n* . Since the answer may be large, **return it modulo** $10^9 + 7$.

A **digit string** is a string consisting of digits 0 through 9 that may contain leading zeros.

Example 1:

Input: $n = 1$

Output: 5

Explanation: The good numbers of length 1 are "0", "2", "4", "6", "8".

Example 2:

Input: $n = 4$

Output: 400

Find how many "good numbers" exist of length n , where:

- Even indices $\rightarrow$ even digits (0,2,4,6,8 $\rightarrow$ 5 choices)
 - Odd indices $\rightarrow$ prime digits (2,3,5,7 $\rightarrow$ 4 choices).
- Return result modulo 10^9+7 .

Pattern

Exponentiation in disguise.

- Even places $\rightarrow 5^{\lceil n/2 \rceil}$
- Odd places $\rightarrow 4^{\lfloor n/2 \rfloor}$.
- Multiply results.

Trick to Remember

👉 Split problem into two **independent exponentiations**.

👉 It's a **direct application of fast power recursion**.

Variations

- Similar to "count binary strings without consecutive 1s" (recurrence).
- Count ways to fill a board with constraints.

✓ C++ Code

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD=1e9+7;
```

```

long long modPow(long long x, long long n) {
    if (n == 0) return 1;
    long long half = modPow(x, n/2) % MOD;
    long long ans = (half * half) % MOD;
    if (n % 2) ans = (ans * x) % MOD;
    return ans;
}

int countGoodNumbers(long long n) {
    long long evenPlaces = (n+1)/2;
    long long oddPlaces = n/2;
    return (modPow(5, evenPlaces) * modPow(4, oddPlaces)) % MOD;
}

int main() {
    cout << countGoodNumbers(50) << endl;
}

```

Complexity

- **Time:** $O(\log n)$ (fast power used)
 - **Space:** $O(\log n)$ recursion
-



Stage 1 Recap (Fun Way)

- **atoi** → “Peel string like onion → 1 digit at a time” 🍷.
- **Pow(x,n)** → “Even halves, odd loses one” ✂️.
- **Good Numbers** → “Even seats 5 guests, odd seats 4 guests → party math 🎉”.



1. Sort a Stack using Recursion



Problem Statement

Given a stack of integers, sort it **in ascending order** using recursion.

👉 No loops, no extra data structures.



Pattern

- **Recursive stack manipulation.**
- Idea: Pop one element, recursively sort remaining stack, insert element back in sorted order.

⚡ Trick to Remember

👉 Think: **Recursion = Bubble Sort + Insertion Sort combined.**

1. Remove top → sort remaining.
2. Insert popped element at **right place** using helper `insertSorted()`.

🤖 Variations

- Sort in **descending order** (just change comparator).
- Sort queue using recursion.

✓ C++ Code

```
#include <bits/stdc++.h>
using namespace std;

void insertSorted(stack<int> &st, int x) {
    if(st.empty() || st.top() <= x) { // insert in correct position
        st.push(x);
        return;
    }
    int temp = st.top(); st.pop();
    insertSorted(st, x);
    st.push(temp);
}

void sortStack(stack<int> &st) {
    if(st.empty()) return;
    int x = st.top(); st.pop();
    sortStack(st); // sort remaining
    insertSorted(st, x);
}

int main(){
    stack<int> st;
    st.push(3); st.push(1); st.push(4); st.push(2);
    sortStack(st);
    while(!st.empty()){ cout<<st.top()<<" "; st.pop(); }
}
```

📊 Complexity

- **Time:** $O(n^2)$ (each insert takes $O(n)$, repeated for n elements).
 - **Space:** $O(n)$ recursion depth.
-

2. Reverse a Stack using Recursion

Problem Statement

Reverse a given stack using recursion.

👉 No loops, no extra stack.

Pattern

- Pop all elements using recursion.
- Insert popped elements **at the bottom** instead of top.

Trick to Remember

👉 Think: **Recursion = pulling out cards and restacking them in reverse.**

Helper `insertAtBottom(x)` places element at bottom.

Variations

- Reverse a queue using recursion.
- Reverse a linked list using recursion (similar thinking).

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

void insertAtBottom(stack<int> &st, int x) {
    if(st.empty()) { st.push(x); return; }
    int temp = st.top(); st.pop();
    insertAtBottom(st, x);
    st.push(temp);
}

void reverseStack(stack<int> &st) {
    if(st.empty()) return;
    int x = st.top(); st.pop();
    reverseStack(st);
    insertAtBottom(st, x);
}

int main(){
    stack<int> st;
    st.push(1); st.push(2); st.push(3); st.push(4);
    reverseStack(st);
    while(!st.empty()){ cout<<st.top()<<" "; st.pop(); }
}
```

Complexity

- **Time:** $O(n^2)$
 - **Space:** $O(n)$ recursion
-

3. Insert at Bottom of Stack

Problem Statement

Insert element x at the **bottom** of stack using recursion.

Pattern

Recursive call removes all elements $\rightarrow$ insert $x \rightarrow$ re-stack them.

Trick to Remember

 “To reach bottom, you must empty the stack.” $\square$

C++ Code

```
void insertAtBottom(stack<int> &st, int x) {  
    if(st.empty()){ st.push(x); return; }  
    int temp = st.top(); st.pop();  
    insertAtBottom(st, x);  
    st.push(temp);  
}
```

Complexity:

- **Time:** $O(n)$
 - **Space:** $O(n)$ recursion
-

4. Delete Middle Element of Stack

Problem Statement

Delete the middle element from stack using recursion.

👉 If size is n , remove $\text{ceil}(n/2)$ -th element.

Pattern

- Use recursion depth to count position.
- Pop elements until middle is reached → skip → push back others.

Trick to Remember

👉 “Middle = vanish, others = reappear.” 🗑️ ✨

✓ C++ Code

```
void deleteMiddle(stack<int> &st, int k) { // k = middle position
    if(k==1) { st.pop(); return; }
    int temp = st.top(); st.pop();
    deleteMiddle(st, k-1);
    st.push(temp);
}

void removeMiddle(stack<int> &st) {
    int n = st.size();
    int mid = (n/2) + 1; // 1-based index
    deleteMiddle(st, mid);
}
```

Complexity

- **Time:** $O(n)$
- **Space:** $O(n)$ recursion



Stage 2 Recap (Fun Memory Notes)

- **Sort Stack** → “Pop, sort, insert like cards in order 📄.”
 - **Reverse Stack** → “Pop all, push at bottom = mirror flip 🔄.”
 - **Insert Bottom** → “Go deep, place x, climb back 📄.”
 - **Delete Middle** → “Magician’s vanish trick 🗑️ ✨.”
-

👉 Would you like me to continue this exact **fun structured note format** for **Stage 3** (String/Sequence Generation problems → Binary Strings, Parentheses, Subsequences) next?

1. Generate All Binary Strings of Length n

Problem Statement

Generate all binary strings of length n .

Ex: $n=3 \rightarrow 000, 001, 010, 011, 100, 101, 110, 111$.

Pattern

Decision Tree → At each index choose 0 or 1.

Trick to Remember

👉 Think: “Every bit has 2 choices → full binary tree with 2^n leaves.”

Variations

- Generate ternary strings (0, 1, 2).
- Generate strings with constraint (like no two consecutive 1s).

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

void generateBinary(int n, string curr="") {
    if(curr.size()==n) { cout<<curr<<"\n"; return; }
    generateBinary(n, curr+"0");
    generateBinary(n, curr+"1");
}

int main(){
    generateBinary(3);
}
```

Complexity

- **Time:** $O(2^n)$
- **Space:** $O(n)$ recursion

Fun Note 🎉: “Each step = coin flip (0 or 1) $\rightarrow$ all coin toss outcomes $\square$.”

2. Generate Valid Parenthesis

Problem Statement

Given n pairs of parentheses, generate all **valid combinations**.

Ex: $n=3 \rightarrow "((()))", "(()())", "(())()", "()(())", "()()()"$.

Pattern

Backtracking with constraints.

- Choices: add (or).
- Constraint: never allow) count > (count.

Trick to Remember

👉 Think: “**Balance** = open count $\geq$ close count always.”

Variations

- Generate valid bracket sequences with {} and [].
- Count #valid sequences (Catalan numbers).

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

void generateParenthesis(int n, int open=0, int close=0, string curr="") {
    if(curr.size()==2*n) { cout<<curr<<"\n"; return; }
    if(open<n) generateParenthesis(n, open+1, close, curr+"(");
    if(close<open) generateParenthesis(n, open, close+1, curr+")");
}

int main(){
    generateParenthesis(3);
}
```

Complexity

- **Time:** $O(2^n)$ but only valid sequences printed (Catalan $\sim 4^n / n^{1.5}$).
- **Space:** $O(n)$ recursion

Fun Note 🎉: “Add (freely,) only if balance allows → Like keeping parentheses wallet 💰.”

3. Print All Subsequences / Power Set

Problem Statement

Given a string/array, print all subsequences (include/exclude each element).

Ex: abc → "", a, b, c, ab, ac, bc, abc.

Pattern

Pick/Not Pick recursion.

Trick to Remember

👉 Every element = **binary choice**: include or exclude.

Variations

- Subset Sum (sum of selected elements).
- Subsequences with constraints (like length ≥ 2).

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

void subseq(string s, int i=0, string curr="") {
    if(i==s.size()) { cout<<curr<<"\n"; return; }
    // pick
    subseq(s,i+1,curr+s[i]);
    // not pick
    subseq(s,i+1,curr);
}

int main(){
    subseq("abc");
}
```

Complexity

- **Time:** $O(2^n)$
- **Space:** $O(n)$ recursion

Fun Note 🎉: “Each element asks $\rightarrow$ ‘Want me or skip me?’ $\rightarrow$ Drama queen 🤩.”

4. Theory: Patterns of Subsequences

There are 3 **Golden Templates** to generate subsequences/sets:

Template 1: Pick/Not Pick

- Use when elements can either be included or excluded.
- Ex: Subsequences, Subset Sum.

Template 2: For Loop Recursion (Combinations)

- Run a loop from $i \rightarrow n$ and recursively pick further elements.
- Ex: Combination Sum, Subset with duplicates.

Template 3: Backtracking with Constraints

- Used when validity depends on conditions (Parentheses, N-Queens).
- Ex: Parenthesis, Sudoku.

Fun Note 🎉:

- Pick/Not Pick = “Yes/No friend invites 🤝.”
 - For-loop recursion = “Team formation one by one 🏆.”
 - Backtracking = “Try until valid, backtrack when wrong 🛑.”
-

Variations (Important for Interviews)

1. **Generate all subsets of a set (with/without duplicates)**
 - With duplicates $\rightarrow$ sort first, skip identical picks.
2. **Generate all strings of length n from given alphabet**
 - Alphabet = $\{ 'a', 'b', 'c' \}$ $\rightarrow$ recursion with multiple choices at each step.

3. **Generate all paths from top-left to bottom-right in a grid**
 - Only moves allowed: Right, Down.
 - Ex: Print all paths in $n \times m$ grid.
-

✓ Example: Generate Strings from Alphabet

```
void generateStrings(int n, string alphabet, string curr="") {
    if(curr.size()==n) { cout<<curr<<"\n"; return; }
    for(char c : alphabet) generateStrings(n, alphabet, curr+c);
}
```

Fun Note 🎉: “Each position = free character slot 📱.”

✓ Example: Paths in Grid

```
void gridPaths(int i, int j, int n, int m, string path="") {
    if(i==n-1 && j==m-1) { cout<<path<<"\n"; return; }
    if(i+1<n) gridPaths(i+1,j,n,m,path+"D");
    if(j+1<m) gridPaths(i,j+1,n,m,path+"R");
}
```

Fun Note 🎉: “At each cell: go Right or Down → maze explorer 🗺️.”



Stage 3 Recap (Memory Hooks)

- **Binary Strings** → “Coin flips ☐.”
- **Parentheses** → “Wallet balance 💰.”
- **Subsequences** → “Drama queen 🧑 (pick or skip).”
- **Theory** → “3 Golden Templates: Yes/No, For-loop team, Validity check.”
- **Grid Paths** → “Explorer 🗺️ Right or Down.”



1. Count All Subsequences with Sum K



Problem Statement

Given an array, count all subsequences whose sum = K.

Pattern

Pick / Not Pick recursion with sum tracking.

Trick to Remember

👉 “Each element asks → include me or leave me? Keep sum book 📖.”

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

int countSubseq(vector<int>& arr, int i, int sum, int target) {
    if(i==arr.size()) return (sum==target);
    // pick
    int left = countSubseq(arr,i+1,sum+arr[i],target);
    // not pick
    int right = countSubseq(arr,i+1,sum,target);
    return left+right;
}

int main(){
    vector<int> arr={1,2,1}; int k=2;
    cout << countSubseq(arr,0,0,k); // 2
}
```

Complexity

- Time: $O(2^n)$
- Space: $O(n)$ recursion

Fun Note 🎉: “Every element is a party invite → you count all friend groups summing to K 🧐.”

2. Check if Subsequence with Sum K Exists

Problem Statement

Return true if **any subsequence** sums to K.

Pattern

Same as above but **return bool early** when found.

Trick

👉 “Stop as soon as one group pays the bill 💵.”

C++ Code

```
bool existsSubseq(vector<int>& arr, int i, int sum, int k) {
    if (sum == k) return true;
    if (i == arr.size()) return false;
    return existsSubseq(arr, i+1, sum+arr[i], k) || existsSubseq(arr, i+1, sum, k);
}
```

Complexity

- Time: $O(2^n)$ worst-case
- Space: $O(n)$

3. Subset Sum I (All subsets)

Problem Statement

Print all subset sums of an array.

Pattern

Pick/Not Pick, but instead of checking target, **store sums**.

C++ Code

```
void subsetSums(vector<int>& arr, int i, int sum, vector<int>& ans) {
    if (i == arr.size()) { ans.push_back(sum); return; }
    subsetSums(arr, i+1, sum+arr[i], ans);
    subsetSums(arr, i+1, sum, ans);
}
```

Fun Note 🎉: “Every path gives one bill total → collect them 📄.”

4. Subset Sum II (Handling Duplicates)

Problem Statement

Return all unique subsets from an array with duplicates.

Pattern

Sort array → **skip duplicates** while picking.

Trick

👉 “Line up identical twins, only let first one talk 😊.”

C++ Code

```
void subset2(int i, vector<int>& arr, vector<int>& temp, vector<vector<int>>& ans) {
    ans.push_back(temp);
    for(int j=i; j<arr.size(); j++){
        if(j>i && arr[j]==arr[j-1]) continue; // skip dups
        temp.push_back(arr[j]);
        subset2(j+1, arr, temp, ans);
        temp.pop_back();
    }
}
```

5. Combination Sum (Unlimited Reuse of Elements)

Problem Statement

Find all combinations of candidates that sum to target (can reuse numbers).

Pattern

For-loop recursion → each element can be reused → don't increment index after picking.

Trick

👉 “Unlimited shopping at same store 🛒.”

✓ C++ Code

```
void combSum(int i, vector<int>& arr, int target, vector<int>& temp,
vector<vector<int>>& ans){
    if(target==0) { ans.push_back(temp); return; }
    if(i==arr.size() || target<0) return;
    // pick (reuse)
    temp.push_back(arr[i]);
    combSum(i, arr, target-arr[i], temp, ans);
    temp.pop_back();
    // not pick
    combSum(i+1, arr, target, temp, ans);
}
```

6. Combination Sum II (Each Element Once, Handle Duplicates)

Problem Statement

Same as above, but each number can be used once, and duplicates avoided.

Pattern

Loop recursion + **skip duplicates** + don't reuse element.

Trick

👉 “One-time coupon 🏠, twins allowed but only once.”

✓ C++ Code

```
void combSum2(int i, vector<int>& arr, int target, vector<int>& temp,
vector<vector<int>>& ans){
    if(target==0){ ans.push_back(temp); return; }
    for(int j=i; j<arr.size(); j++){
        if(j>i && arr[j]==arr[j-1]) continue;
        if(arr[j]>target) break;
        temp.push_back(arr[j]);
        combSum2(j+1, arr, target-arr[j], temp, ans);
        temp.pop_back();
    }
}
```

7. Combination Sum III

Problem Statement

Find all combinations of k numbers (from 1–9) that sum to n .

Pattern

For-loop recursion with constraints k & n .

Trick

👉 “Pick exactly k kids whose chocolates sum to n 🍫.”

C++ Code

```
void combSum3(int start,int k,int n,vector<int>& temp,vector<vector<int>>& ans){
    if(k==0 && n==0){ ans.push_back(temp); return; }
    if(k==0 || n<=0) return;
    for(int i=start;i<=9;i++){
        temp.push_back(i);
        combSum3(i+1,k-1,n-i,temp,ans);
        temp.pop_back();
    }
}
```

Variations (Very Common in Interviews)

1. **Partition Equal Subset Sum (Leetcode 416)**
→ Check if array can be partitioned into 2 subsets with equal sum.
Trick: Target = totalSum/2 → use recursion/DP.
 2. **Minimum Subset Sum Difference**
→ Partition into 2 subsets with min $|S1 - S2|$.
Trick: Explore subset sums $\leq \text{total}/2$.
 3. **Subset sum with multiplicities**
→ Each element can be picked up to k times.
 4. **K-subset partitioning**
→ Divide array into k subsets with equal sum.
Trick: Backtracking with bucket filling.
-



Stage 4 Recap (Memory Hooks)

- Count subseq = all party groups paying bill 🧑‍🤝‍🧑
- Exists subseq = stop once one pays 🏠
- Subset Sum = collect all bill totals 📄
- Subset Sum II = twins but skip duplicates 🧑‍🧑
- Comb Sum = unlimited shopping 🛒
- Comb Sum II = one-time coupon 🎫
- Comb Sum III = pick k kids with chocolates 🍫
- Partition Equal Subset Sum = split pizza equally 🍕



5. Recursion with String/Combination Building

👉 **Goal:** Recursively build **strings/expressions/partitions** → DFS style.
 Think: **Pick a character** → **build partial string** → **recurse** → **backtrack**.



Problem 1: Letter Combinations of a Phone Number

Statement: Given a digit string, return all possible letter combinations using phone keypad mapping.

Pattern: Digit → map to chars → DFS to build all strings.

Trick: Treat like a tree: digit = level, char = branch.

Clean Code:

```
vector<string> res;
vector<string> mp = {"", "", "abc", "def", "ghi", "jkl", "mno", "pqrs",
"tuv", "wxyz"};

void dfs(string &digits, int idx, string curr) {
    if(idx == digits.size()) { res.push_back(curr); return; }
    for(char c : mp[digits[idx]-'0'])
        dfs(digits, idx+1, curr+c);
}

vector<string> letterCombinations(string digits) {
    if(digits.empty()) return {};
    dfs(digits, 0, "");
    return res;
}
```

TC: $O(4^n)$ (worst case, each digit has 4 letters).
SC: $O(n)$ recursion depth.

✂ **Trick to remember: Digit** → **branch out letters**. Each digit = **level in recursion tree**.

✓ Problem 2: Palindrome Partitioning

Statement: Partition string into substrings, all palindromes.

Pattern: DFS cut & check palindrome → backtrack.

Trick: “At each index → cut or not cut → recurse.”

Clean Code:

```
vector<vector<string>> res;

bool isPal(string &s, int l, int r) {
    while(l < r) if(s[l++] != s[r--]) return false;
    return true;
}

void dfs(string &s, int idx, vector<string> &path) {
    if(idx == s.size()) { res.push_back(path); return; }
    for(int end = idx; end < s.size(); end++) {
        if(isPal(s, idx, end)) {
            path.push_back(s.substr(idx, end - idx + 1));
            dfs(s, end + 1, path);
            path.pop_back();
        }
    }
}

vector<vector<string>> partition(string s) {
    vector<string> path;
    dfs(s, 0, path);
    return res;
}
```

TC: $O(2^n \cdot n)$ (check palindrome).

SC: $O(n)$.

✂ **Trick:** “Cut string like samosa → only keep if piece = palindrome.”

✓ Problem 3: Word Break (Leetcode 139)

Statement: Given string s + dict $\rightarrow$ can you break into valid words?

Pattern: DFS + memo (or DP).

Trick: “At index $\rightarrow$ check all prefixes $\rightarrow$ recurse if valid.”

Clean Code:

```
unordered_set<string> dict;
unordered_map<int,bool> dp;

bool dfs(string &s,int idx){
    if(idx==s.size()) return true;
    if(dp.count(idx)) return dp[idx];
    for(int end=idx; end<s.size(); end++){
        if(dict.count(s.substr(idx,end-idx+1)) && dfs(s,end+1))
            return dp[idx]=true;
    }
    return dp[idx]=false;
}

bool wordBreak(string s, vector<string>& wordDict){
    dict={wordDict.begin(),wordDict.end()};
    return dfs(s,0);
}
```

TC: $O(n^2)$.

SC: $O(n)$.

✂ **Trick:** “Try every prefix like chopping cake $\rightarrow$ if word in dict $\rightarrow$ recurse on rest.”

✓ Problem 4: Expression Add Operators

Statement: Insert +, -, * between digits to evaluate to target.

Pattern: DFS with carry state: index, expr, eval, prevNum.

Trick: Carry previous number for multiplication correction.

Clean Code:

```
vector<string> res;

void dfs(string &num,int idx,long eval,long prev,string expr,int target){
    if(idx==num.size()){
        if(eval==target) res.push_back(expr);
        return;
    }
    for(int i=idx;i<num.size();i++){
        if(i>idx && num[idx]=='0') break; // no leading zero
```

```

        string part=num.substr(idx,i-idx+1);
        long cur=stol(part);
        if(idx==0) dfs(num,i+1,cur,cur,part,target);
        else{
            dfs(num,i+1,eval+cur,cur,expr+" "+part,target);
            dfs(num,i+1,eval-cur,-cur,expr+"-"+part,target);
            dfs(num,i+1,eval-prev+prev*cur,prev*cur,expr+"*"+part,target);
        }
    }
}

vector<string> addOperators(string num,int target){
    dfs(num,0,0,0,"",target);
    return res;
}

```

TC: $O(4^n)$.

SC: $O(n)$.

📌 **Trick:**

- Keep track of `eval` (expression result so far).
- Keep `prev` (last number for handling `*`).
- Always **undo multiplication** by `eval - prev + prev*cur`.

Variations (Interview Favourites)

1. **Restore IP Addresses (LC 93):** DFS, cut string into 4 valid segments.
2. **Decode Ways (LC 91):** Similar to word break → ways to split digits.
3. **Generate valid expressions (parentheses, operators):** Same DFS pattern.

Pattern Memory Hack

Think **String Recursion = DFS + Cut & Build**:

- Digits → branch out chars (**Letter Combos**).
- String → cut into valid partitions (**Palindrome, Word Break**).
- Numbers → insert operators (**Expression Add**).

📌 Mnemonic: “**D**igits → **E**xpand, **S**tring → **C**ut, **N**umbers → **I**nsert.”

6. Recursion with Grids / Boards (Backtracking)

Core Idea:

You have a grid/board. You recursively **explore cells/positions**, and backtrack when invalid. Think of it as:

DFS + “Mark → Explore → Unmark” (Undo).

1. Word Search (Leetcode 79)

Problem: Given a 2D board and a word, check if the word exists in the grid by moving up/down/left/right.

Pattern: DFS on grid + backtracking.

Trick to Remember:

“Mark → Search deeper → Unmark”

Imagine writing the word letter by letter on the board and erasing if dead-end.

Code:

```
bool dfs(vector<vector<char>>& board, string& word, int i, int j, int k) {
    if (k == word.size()) return true;
    if (i<0 || j<0 || i>=board.size() || j>=board[0].size() || board[i][j] !=
word[k])
        return false;

    char temp = board[i][j];
    board[i][j] = '#'; // mark
    bool found = dfs(board, word, i+1, j, k+1) || dfs(board, word, i-1, j,
k+1) ||
                dfs(board, word, i, j+1, k+1) || dfs(board, word, i, j-1,
k+1);
    board[i][j] = temp; // unmark
    return found;
}

bool exist(vector<vector<char>>& board, string word) {
    for (int i=0;i<board.size();i++)
        for (int j=0;j<board[0].size();j++)
            if (dfs(board, word, i, j, 0)) return true;
    return false;
}
```

Complexity: $O(N * 4^L)$ (N = cells, L = word length).

 Ok for interviews.

✓ 2. N-Queens

Problem: Place N queens on NxN board such that no two attack each other.

Pattern: Try row by row. Place queen → move next row → backtrack if invalid.

Trick:

"Think: Rows = recursion levels, Columns = choices."

Use col, diag1, diag2 sets to check safety.

Code:

```
vector<vector<string>> res;

void solve(int n, int row, vector<int>& cols, vector<int>& d1, vector<int>& d2, vector<string>& board) {
    if (row == n) { res.push_back(board); return; }
    for (int col=0; col<n; col++) {
        if (cols[col] || d1[row-col+n] || d2[row+col]) continue;
        cols[col] = d1[row-col+n] = d2[row+col] = 1;
        board[row][col] = 'Q';
        solve(n, row+1, cols, d1, d2, board);
        board[row][col] = '.';
        cols[col] = d1[row-col+n] = d2[row+col] = 0;
    }
}

vector<vector<string>> solveNQueens(int n) {
    vector<string> board(n, string(n, '.'));
    vector<int> cols(n,0), d1(2*n,0), d2(2*n,0);
    solve(n,0,cols,d1,d2,board);
    return res;
}
```

Complexity: O(N!).

👉 Classic Amazon/Google question.

✓ 3. Rat in a Maze

Problem: Find all paths from (0,0) to (n-1,n-1).

Pattern: DFS on grid, only move valid cells.

Trick:

"D → L → R → U order, mark path."

Code:

```

void dfsMaze(vector<vector<int>>& m, int x, int y, string path,
vector<string>& ans) {
    int n = m.size();
    if (x==n-1 && y==n-1) { ans.push_back(path); return; }
    if (x<0 || y<0 || x>=n || y>=n || m[x][y]==0) return;
    m[x][y]=0; // mark
    dfsMaze(m,x+1,y,path+'D',ans);
    dfsMaze(m,x,y-1,path+'L',ans);
    dfsMaze(m,x,y+1,path+'R',ans);
    dfsMaze(m,x-1,y,path+'U',ans);
    m[x][y]=1; // unmark
}

```

Complexity: $O(4^{(N^2)})$.

✓ 4. Sudoku Solver

Problem: Fill board so every row/col/3x3 block has 1–9.

Pattern: Try digit → check safe → recurse.

Trick:

"Think cell by cell: Try → Check → Move → Backtrack."

Code (short):

```

bool solve(vector<vector<char>>& board) {
    for (int i=0;i<9;i++) {
        for (int j=0;j<9;j++) {
            if (board[i][j]=='.') {
                for (char c='1'; c<='9'; c++) {
                    bool ok = true;
                    for (int k=0;k<9;k++)
                        if (board[i][k]==c || board[k][j]==c ||
board[3*(i/3)+k/3][3*(j/3)+k%3]==c) ok=false;
                    if (ok) {
                        board[i][j]=c;
                        if (solve(board)) return true;
                        board[i][j]='.';
                    }
                }
                return false;
            }
        }
    }
    return true;
}

void solveSudoku(vector<vector<char>>& board) { solve(board); }

```

✓ 5. M Coloring Problem

Problem: Given graph + M colors, check if you can color such that no adjacent share color.

Pattern: Try node → assign color → recurse.

Trick:

"DFS on graph, like N-Queens but on adjacency."

Code:

```
bool safe(int node, int c, vector<int>& color, vector<vector<int>>& graph) {
    for (int nb: graph[node]) if (color[nb]==c) return false;
    return true;
}
bool dfsColor(int node, vector<int>& color, vector<vector<int>>& graph, int m) {
    if (node==graph.size()) return true;
    for (int c=1;c<=m;c++) {
        if (safe(node,c,color,graph)) {
            color[node]=c;
            if (dfsColor(node+1,color,graph,m)) return true;
            color[node]=0;
        }
    }
    return false;
}
```



Variations (You must master for Amazon/Google/MSFT):

1. **Knight's Tour Problem** – Same as Rat in Maze but knight moves.
 2. **Unique Paths (with obstacles)** – Count paths using recursion/DP.
 3. **Hamiltonian Path** – Same as M coloring but instead of coloring, try to visit all nodes once.
-



Memory Hooks

- **Word Search** → "Write & Erase letters."
- **N-Queen** → "Row = Level, Col = Choice, Diagonal = Restriction."

- **Rat Maze** → “DLRU → Direction order like compass.”
- **Sudoku** → “Try a number → Move ahead → Backtrack.”
- **M Coloring** → “Graph like Queens but edges restrict colors.”